

小车巡线

小车巡线

- 1. 快速上手
- 2. 硬件接线
- 3. 使用方法
- 4. 现象结果
- 5. 代码解释

1. 快速上手

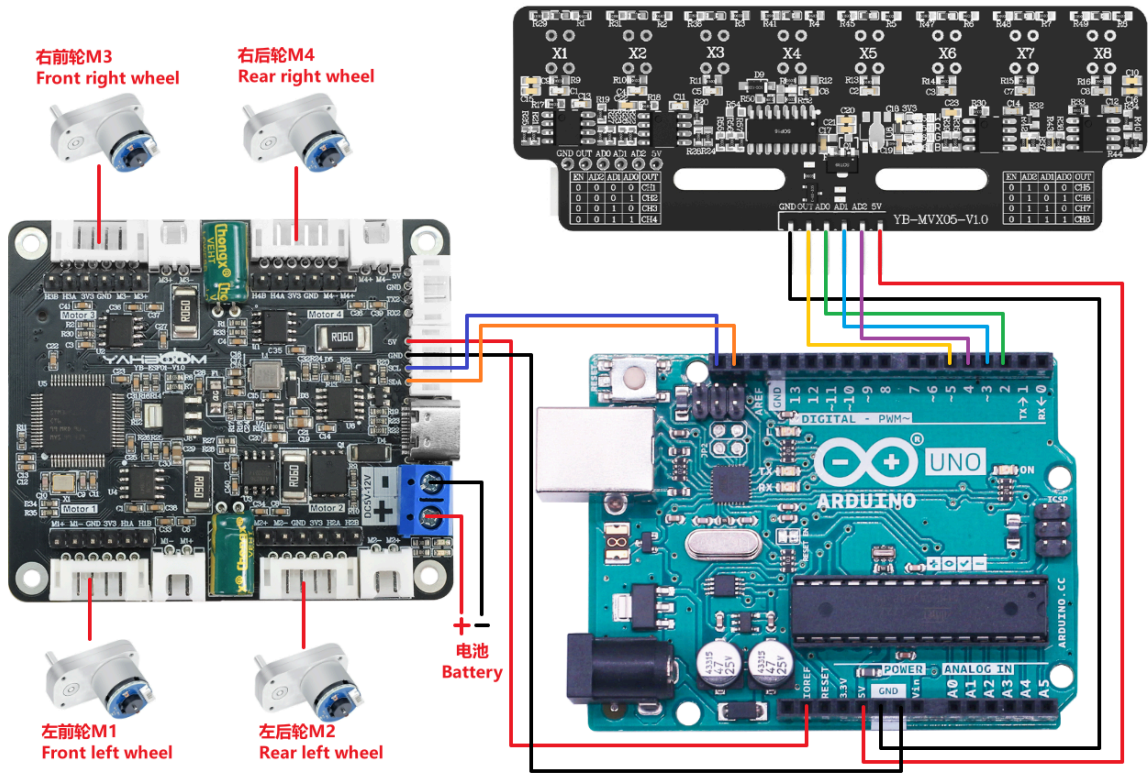
本教程讲解如何使用 Arduino UNO 主板在获取八路灰度巡线模块的数据之后，如何修改代码中的部分参数，使小车在用户自己的赛道场景中运行的效果更好。

教程中使用的硬件是Arduino UNO、亚博售卖的八路灰度巡线模块、轮式小车底盘V2、L型520电机、四路电机驱动板、12V电池组，查看下方的接线图进行接线后，再给 Arduino UNO 烧录程序，就可以上电开始巡线了。如果效果不佳，可以跳转到代码讲解章节，对巡线参数进行修改，优化巡线效果。

除此之外，还需要修改合适的电机PID参数，本案例中使用的是L型520电机，使用的电机PID值为：P:1.9,I:0.2,D:0.8。使用PC串口助手与四路电机驱动板通信，发送指令修改PID值，如果使用的是其他电机，需要自行测试出一个合适的电机PID参数。

此调优步骤需要用户具备一定的调试小车经验。非亚博的模块仅供参考。

2. 硬件接线



八路灰度巡线模块	Arduino UNO
5V	5V

八路灰度巡线模块	Arduino UNO
GND	GND
AD0	2
AD1	3
AD2	4
OUT	5

Arduino UNO	四路电机驱动板
IOREF(5V)	5V
GND	GND
SCL	SCL
SDA	SDA

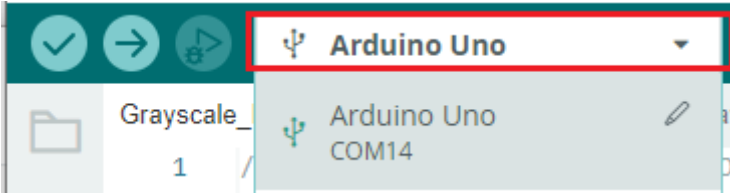
亚博售卖的八路灰度巡线模块标配的线材为XH2.54转6pin杜邦线，XH2.54排线接口一端要接在八路灰度模块上，另一端的杜邦线则与上图一样正常连接即可：



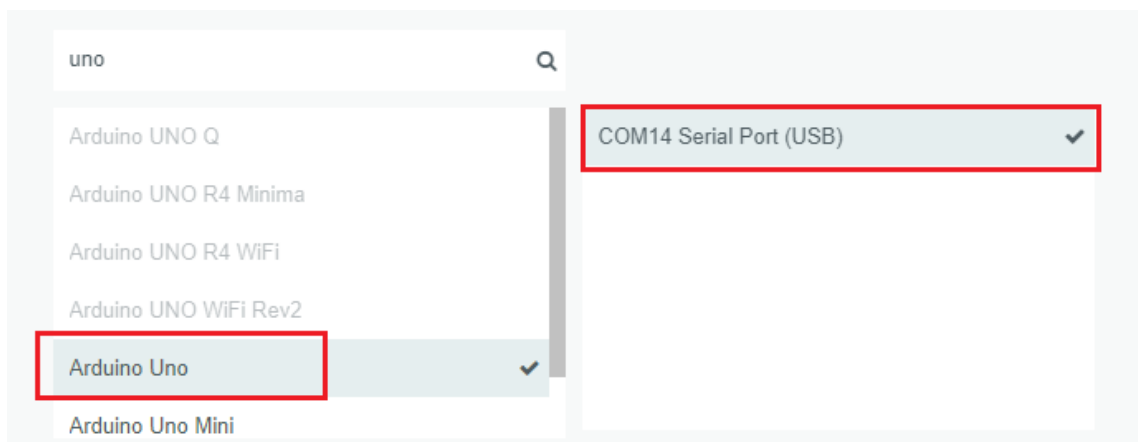
亚博售卖的四路电机驱动板与L型520电机的连接方法可以到四路电机驱动板资料中找到《常用电机介绍以及用法》的教程，查看详细的具体接线方式，由于篇幅较长，这里不再展开。

3. 使用方法

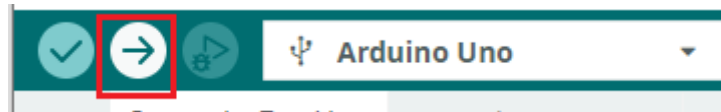
1. 使用 Arduino IDE 打开 Grayscale_Trace 文件夹下的 Grayscale_Trace.ino 文件。
2. 通过数据线将 Arduino UNO 连接到电脑。
3. 在 Arduino IDE 的左上角的菜单栏中，点击端口选择框，然后选择你接入的arduino的串口。



4. 如果首次是首次连接有COM端口号，但是没有arduino型号，则手动选择型号



5. 接着点击工具栏上的 "上传" (Upload) 按钮，程序将开始编译并烧录到 Arduino UNO 中。



6. 烧录完成后，断开 Arduino 与电脑的连接，将小车放在赛道上，为小车接上电池电源。

4. 现象结果

上电之后，程序开始运行。此时如果八路灰度巡线模块上的八个LED灯都是全亮或者全灭，小车则不会动，防止因为被放在了错误的地图上乱跑。当巡线模块上的灯不一致时，小车开始巡线。

在要巡的线上的巡线模块LED灯，需要和线外的赛道相反，也就是说假如线上的LED灯是灭的，线外面的赛道上对应的LED灯需要亮起，才能正常巡线。

5. 代码解释

```
#####  
// 重要配置参数 Important Configuration Parameters  
#####  
// 根据自己使用的电机类型，选择对应的电机参数配置  
// Select the corresponding motor parameter configuration according to the type  
of motor you use  
#define MOTOR_TYPE 5 //1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520  
电机  
//1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC  
reduction motor 5:L type 520 motor  
#####  
// 将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则数值为1，LINE_RAW_VALUE=1；如果灯  
灭，则数值为0，LINE_RAW_VALUE=0  
// Place the grayscale line-following module on the track and observe the light  
facing the line. If the light is on, the value is 1, LINE_RAW_VALUE=1; if the  
light is off, the value is 0, LINE_RAW_VALUE=0  
#define LINE_RAW_VALUE 1  
#####
```

代码开头的地方有 `MOTOR_TYPE` 和 `LINE_RAW_VALUE` 两个宏定义设置。

- `MOTOR_TYPE`：根据自己所购买的电机修改 `MOTOR_TYPE`，代码中适配的电机都是亚博目前在售的电机。
- `LINE_RAW_VALUE`：需要将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则填 `LINE_RAW_VALUE=1`；如果灯灭，则数值为0，`LINE_RAW_VALUE=0`。

```
//Grayscale_Trace.ino
void line_following_init(line_following_t* controller) {
    controller->kp = 100.0f;    // 比例系数 | proportional gain
    controller->ki = 0.5f;      // 积分系数 | integral gain
    controller->kd = 50.0f;      // 微分系数 | derivative gain

    controller->last_error = 0.0f; // 上次偏差 | last error
    controller->integral = 0.0f;    // 积分累积 | integral accumulation

    controller->base_speed = 350;    // 基础速度 | base speed
    controller->max_speed = 700;     // 最大速度 | maximum speed

    controller->sensor_weights[0] = -5.0f;
    controller->sensor_weights[1] = -4.0f;
    controller->sensor_weights[2] = -2.0f;
    controller->sensor_weights[3] = -1.0f;
    controller->sensor_weights[4] = 1.0f;
    controller->sensor_weights[5] = 2.0f;
    controller->sensor_weights[6] = 4.0f;
    controller->sensor_weights[7] = 5.0f;

    controller->motor_locked = true; // 上电默认锁定电机 | lock motors on power-up
}
```

优化巡线效果重点修改项：

PID 参数

kp: 调大 **kp** 可使响应更快，但易震荡；调小 **kp** 可减少震荡，但响应会变慢且可能偏离路线。

ki: 调大 **ki** 可更快消除稳态误差，但易引起超调和震荡；调小 **ki** 可降低超调风险，但消除误差变慢甚至无法完全消除。

kd: 调大 **kd** 可提升系统稳定性，减少摆动和超调，但过大会对噪声敏感并引发抖动；调小 **kd** 可降低对噪声的敏感，但抑制摆动能力减弱，响应可能滞后或超调。

速度参数

base_speed: 决定小车在平稳巡线时的前进速度，影响整体效率和弯道响应。

max_speed: 限制小车左右轮的最高转速，防止小车加速偏差过大、过快。

灰度巡线模块权重参数

sensor_weights: 修改灰度巡线模块上的权重，第一个值指代模块最边缘的X1灯，第二个值为X2灯。数值调大，则在X1灯亮时反应更剧烈，调小则平缓。

```
// bsp_motor_iic.cpp
void Set_Motor(int MOTOR_TYPE)
{
    if(MOTOR_TYPE == 1)
    {
        Set_motor_type(1);
    }
}
```

```

        delay(100);
        Set_Pluse_Phase(30);
        delay(100);
        Set_Pluse_line(11);
        delay(100);
        Set_wheel_dis(67.00);
        delay(100);
        Set_motor_deadzone(1900);
        delay(100);
    }
    //...
}

void control_speed(int16_t m1,int16_t m2 ,int16_t m3,int16_t m4)
{
    static uint8_t speed[8];
    speed[0] = (m1>>8)&0xff;
    speed[1] = (m1)&0xff;
    //...
    i2cwrite(Motor_model_ADDR,SPEED_Control_REG,8,speed);
}

```

- `Set_Motor`: 根据预设的电机类型 (MOTOR_TYPE) , 配置电机的相关参数 (类型、减速比、磁环线、轮径、死区) 。
- `control_speed` : 通过I2C向电机驱动板发送指令, 以编码器闭环控制四个电机的速度。

```

// grayscale_sensor.cpp
void Grayscale_Sensor_Read_All(uint16_t* sensor_values)
{
    uint8_t i;
    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        _select_channel(i);
        delayMicroseconds(50);
        sensor_values[i] = Read_OUT_value();
    }
}

```

- `Grayscale_Sensor_Read_All` : 依次选择所有传感器通道, 读取并返回每个通道的输出值列表。

```

// trace_task.cpp
void follow_line(line_following_t* controller, uint16_t* sensor_values,
                uint8_t motor_type, uint16_t line_raw_value)
{
    if (controller->motor_locked) {
        if (check_sensors_safe(controller, sensor_values)) {
            controller->motor_locked = false;
        } else {
            control_pwm(0, 0, 0, 0);
            return;
        }
    }
}

```

```

    }
}

float error = calculate_error(controller, sensor_values, line_raw_value);
float pid_output = pid_control(controller, error);

int16_t left_speed, right_speed;
differential_speed_control(controller, pid_output, &left_speed,
&right_speed);

if (motor_type == 1 || motor_type == 2 || motor_type == 3 || motor_type == 5)
{
    control_speed(left_speed, left_speed, right_speed, right_speed);
} else if (motor_type == 4) {
    control_pwm(left_speed, right_speed, left_speed, right_speed);
}
}

```

- `check_sensors_safe` : 检查所有灰度传感器是否都显示相同值，用于在启动时判断小车是否处于安全状态。
- `calculate_error` : 根据灰度传感器读取的值和预设权重，计算小车偏离线中心的误差值。
- `pid_control` : 执行PID控制算法，根据输入的误差值计算控制输出。
- `differential_speed_control` : 根据PID控制输出，计算左右轮的差速，从而调整小车转向。
- `follow_line` : 巡线主函数，负责读取传感器、执行安全检查、计算误差、进行PID控制和差速控制来驱动电机。